



数据库系统

第三章 关系数据库标准语言SQL

胡金鹏



SQL语言



3.1 SQL概述

3.2 数据定义

3.3 数据查询

3.4 数据更新

3.5 视图

- ☞ SQL: Structured Query Language , **结构化查询语言** , 是**关系数据库的标准语言**
- ☞ IBM , Sequel语言(Structured English Query Language) ; 后改名为 SQL(Structured Query Language) ;
- ☞ ANSI 和 ISO 发布的 SQL标准:
 - SQL-86、SQL-89、SQL-92 (SQL2) ;
 - SQL:1999 (SQL3)
 - SQL:2003 , SQL:2008 , SQL:2011 , SQL:2016
- ☞ SQL是关系数据库的标准语言 , 是一个通用的、功能极强的关系数据库语言。但也应注意不同软件商的数据库产品满足的标准可能是不一样的 , 具体应用还是略有差异。

SQL的特点

一、综合统一

- ✓ 集**数据定义语言**（ DDL ）， **数据操纵语言**（ DML ）、 **数据控制语言**（ DCL ）功能于一体
- ✓ 可以独立完成数据库生命周期中的全部活动：定义模式关系，插入数据，建立数据库；对数据库中的数据进行查询和更新；数据库重构和维护；数据库安全性、完整性控制
- ✓ 用户数据库投入运行后，可根据需要随时逐步修改模式，不影响数据的运行
- ✓ 数据操作符统一

SQL的特点

二、高度非过程化

- ✓ 非关系数据模型的数据操纵语言“面向过程”，必须制定存取路径
- ✓ SQL只要提出“做什么”，无须了解存取路径
- ✓ 存取路径的选择以及SQL的操作过程由系统自动完成

SQL的特点

☞ 三、面向集合的操作方式

- ✓ 非关系数据模型采用面向记录的操作方式，操作对象是一条记录
- ✓ SQL采用集合操作方式：操作对象、查找结果可以是元组的集合；一次插入、删除、更新操作的对象可以是元组的集合

SQL的特点

四、以同一种语法结构提供多种使用方式

- ✓ SQL是独立的语言，能够独立第用于联机交互的使用方式
- ✓ SQL又是嵌入式语言，能够嵌入到高级语言（如C，C++，Java）程序中，供程序员设计程序时使用。



SQL的特点

👉 五、语言简洁，易学易用

- ✓ SQL完成核心功能只用了9个动词

SQL 蹕刎鲸罇踉

SQL 功 能	动 词
数 据 查 询	SELECT
数 据 定 义	CREATE, DROP, ALTER
数 据 操 纵	INSERT, UPDATE DELETE
数 据 控 制	GRANT, REVOKE

SQL的特点

👉 小结

综合统一：SQL语言集功能于一体，语言风格统一

高度非过程化：路径选择及处理过程由系统自动完成

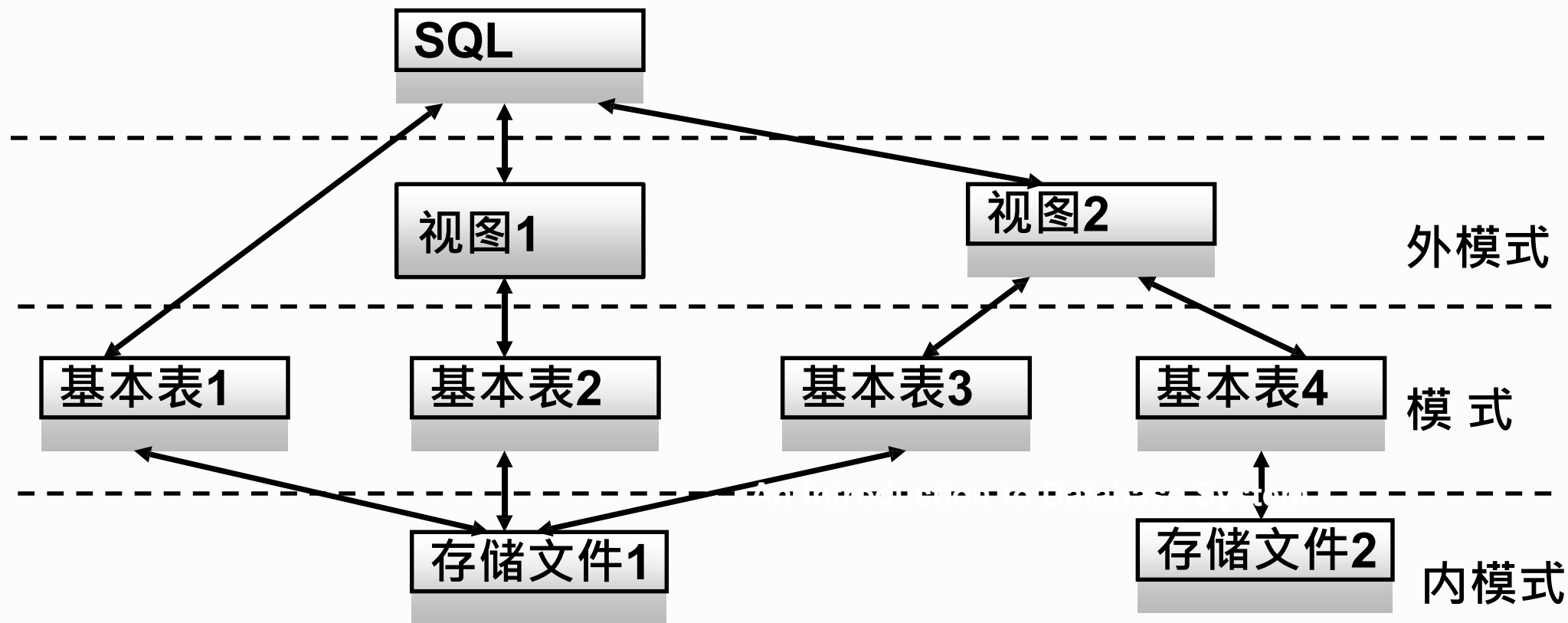
面向集合的操作方式：操作对象都可以是元组的集合

同一语法结构提供两种使用方式：键盘输入/嵌入

语言简捷，易学易用：九条核心语句，结构化特性



SQL支持关系数据库三级模式结构



- 基本表

- 本身独立存在的表
- SQL中一个关系就对应一个基本表
- 一个(或多个)基本表对应一个存储文件
- 一个表可以带若干索引

- 存储文件

- 逻辑结构组成了关系数据库的内模式
- 物理结构是任意的，对用户透明

- 视图

- 从一个或几个基本表导出的表
- 数据库中只存放视图的定义而不存放视图对应的数据
- 视图是一个虚表
- 用户可以在视图上再定义视图



SQL语言



3.1 SQL概述

3.2 数据定义

3.3 数据查询

3.4 数据更新

3.5 视图



SQL的数据定义功能: 模式定义、表定义、视图和索引的定义

操 作 对 象	操 作 方 式		
	创 建	删 除	修 改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视 图	CREATE VIEW	DROP VIEW	
索 引	CREATE INDEX	DROP INDEX	



数据定义



1 模式的定义与删除

2 基本表的定义、删除与修改

3 索引的建立与删除

一、定义模式

- 定义模式实际上定义了一个**命名空间**
- 在这个空间中可以定义该模式包含的数据库对象，例如基本表、视图、索引等。模式定义语句：

CREATE SCHEMA <模式名> AUTHORIZATION <用户名>

【例】为用户WANG定义一个学生-课程模式S-T

CREATE SCHEMA "S-T" AUTHORIZATION WANG

【说明】若没有指定模式名，那么<模式名>隐含为<用户名>

CREATE SCHEMA AUTHORIZATION WANG

一、定义模式

- 在CREATE SCHEMA中可以接受CREATE TABLE，CREATE VIEW和GRANT子句

CREATE SCHEMA <模式名> AUTHORIZATION <用户名> [<表定义子句> | <视图定义子句> | <授权定义子句>]

【例】为用户ZHANG创建一个模式TEST，并在其中并在其中定义了一个表TAB1

CREATE SCHEMA TEST AUTHORIZATION ZHANG

CREATE TABLE TAB1(COL1 SMALLINT ,

COL2 INT ,

COL3 CHAR(20) ,

COL4 NUMERIC(10 , 3) ,

COL5 DECIMAL(5 , 2)

);

TAB1 属于TEST模式



一、定义模式

- 执行创建模式语句必须拥有DBA权限，或者DBA授予某一个用户在CREATE SCHEMA的权限。**该用户必须实际存在**

二、删除模式

- 语句格式

DROP SCHEMA <模式名> <CASCADE|RESTRICT>

【说明】 CASCADE和RESTRICT必须二选一

CASCADE（级联）：删除模式的同时把改模式中所有的数据库对象全部删除

RESTRICT（限制）：如果该模式中定义了下属的数据库对象（如表、视图等），则**拒绝**该删除语句的执行。当该模式中没有任何下属的对象时才能执行

【例】 DROP SCHEMA TEST CASCADE

删除模式TEST，同时该模式中定义的表TAB1也被删除



数据定义



1 模式的定义与删除

2 基本表的定义、删除与修改

3 索引的建立与删除



一、定义基本表

- 语句格式

CREATE TABLE <表名>

(<列名> <数据类型> [列级完整性约束条件]

[,<列名> <数据类型> [列级完整性约束条件]]

.....

[,<表级完整性约束条件>]) ;



一、定义基本表

- 完整性规则主要有三种子句：

主键子句(PRIMARY KEY)：实体完整性

外键子句(FOREIGN KEY)：参照完整性

检查子句(CHECK)：用户定义完整性

【说明】完整性约束条件涉及到该表的多个属性，则必须定义在表级上，否则既可以定义在列级也可以定义在表级。



一、定义基本表

- 学生表：Student (Sno , Sname , Sex , Sage , Sdept)
- 课程表：Course (Cno, Cname, Ccredit, Cpno)
- 学生选课表：SC(Sno, Cno, Grade)

一、定义基本表

【例】建立“学生”表Student，学号是主码，姓名取值唯一。

```
CREATE TABLE Student
```

```
(Sno CHAR(9) PRIMARY KEY , /* 列级完整性约束条件*/
```

```
Sname CHAR(20) UNIQUE , /* Sname取唯一值*/
```

```
Ssex CHAR(2) , /*一个汉字两个字符*/
```

```
Sage SMALLINT ,
```

```
Sdept CHAR(20)
```

```
);
```

主码



一、定义基本表

【例】建立一个“课程”表Course

```
CREATE TABLE Course
```

```
( Cno CHAR(4) PRIMARY KEY ,
```

```
  Cname CHAR(40) ,
```

```
  Cpno CHAR(4) ,
```

```
  Ccredit SMALLINT ,
```

```
  FOREIGN KEY (Cpno) REFERENCES Course(Cno)
```

```
);
```

Cpno是外码
被参照表是Course
被参照列是Cno

一、定义基本表

【例】建立一个“学生选课”表SC

```
CREATE TABLE SC
(Sno CHAR(9) ,
Cno CHAR(4) ,
Grade SMALLINT ,
PRIMARY KEY (Sno , Cno) ,
/* 主码由两个属性构成，必须作为表级完整性进行定义*/
FOREIGN KEY (Sno) REFERENCES Student(Sno) ,
/* 表级完整性约束条件，Sno是外码，被参照表是Student */
FOREIGN KEY (Cno) REFERENCES Course(Cno)
/* 表级完整性约束条件，Cno是外码，被参照表是Course*/
);
```

一、定义基本表

【例】建立一个“学生选课”表SC

```
CREATE TABLE SC
(Sno CHAR(9) ,
Cno CHAR(4) ,
Grade SMALLINT CHECK (Grade≤100) ,
PRIMARY KEY (Sno , Cno) ,
FOREIGN KEY (Sno) REFERENCES Student(Sno) ,
FOREIGN KEY (Cno) REFERENCES Course(Cno)
/* 表级完整性约束条件，Cno是外码，被参照表是Course*/
);
```

用户定义完整性，
检查子句
(CHECK)



数据类型

- SQL中域的概念用**数据类型**来实现
- 定义表的属性时 需要指明其数据类型及长度
- 选用哪种数据类型
 - 取值范围
 - 要做哪些运算



数据定义



数据类型	含义
CHAR(n)	长度为n的定长字符串
VARCHAR(n)	最大长度为n的变长字符串
INT	长整数（也可以写作INTEGER）
SMALLINT	短整数
NUMERIC(p, d)	定点数，由p位数字（不包括符号、小数点）组成，小数后面有d位数字 例如： (10, 3) 7654321.123
REAL	取决于机器精度的浮点数
Double Precision	取决于机器精度的双精度浮点数
FLOAT(n)	浮点数，精度至少为n位数字
DATE	日期，包含年、月、日，格式为YYYY-MM-DD
TIME	时间，包含一日的时、分、秒，格式为HH:MM:SS



■目前SQL支持的常见数据类型

- ✓字符型：char(n) , varchar(n).
- ✓数值型：int , smallint , numeric(p,d) , real , double precision , float(n).
- ✓空值：Null
- ✓日期时间：date , time , timestamp , Interval
- ✓大对象类型：clob
- ✓用户自定义类型：
create domain , create type



模式与表

- 每一个基本表都属于某一个模式
- 一个模式包含多个基本表
- 定义基本表时，若没有指定所属模式，系统会根据搜索路径来确定所属模式

显示当前搜索路径：`SHOW search_path`

搜索路径当前默认值是：`$user, PUBLIC;`

DBA用户可以设置搜索路径：`SET search_path TO "S-T" , PUBLIC;`



模式与表

- DBA用户可以设置搜索路径，然后定义基本表

SET search_path TO “S-T”, PUBLIC ;

Create table Student (.....) ;

结果建立了S-T.Student基本表。

RDBMS发现搜索路径中第一个模式名S-T存在，就把该模式作为基本表Student所属的模式。

模式与表

- 定义基本表所属模式

- **方法一**：在表名中明显地给出模式名

- ```
Create table "S-T".Student (.....); /*模式名为 S-T*/
```

- ```
Create table "S-T".Course ( ..... );
```

- ```
Create table "S-T".SC (.....);
```

- **方法二**：在创建模式语句中同时创建表

- **方法三**：设置所属的模式

- ```
SET search_path TO "S-T" , PUBLIC;
```

- ```
Create table Student (.....)
```

- 建立了S-T.Student 基本表





## 二、修改基本表

- 语句格式

**ALTER TABLE <表名>**

**[ ADD <新列名> <数据类型> [ 完整性约束 ] ]**

**[ADD <表级完整性约束>]**

**[ DROP [COLUMN] <列名> [CASCADE|RESTRICT]]**

**[ DROP CONSTRAINT <完整性约束名> [CASCADE|RESTRICT]]**

**[ ALTER COLUMN<列名> <数据类型> ] ;**



## 二、修改基本表

- **<表名>是要修改的基本表**
- **ADD子句用于增加新列、新的列级完整性约束和表级完整性约束条件**
- **DROP COLUMN子句用于删除表中的列**
  - 如果指定了CASCADE短语，则自动删除引用该列的其他对象
  - 如果指定了RESTRICT短语，则如果该列被其他对象引用，则数据库管理系统拒绝删除该列
- **DROP CONSTRAINT子句用于删除指定的完整性约束条件**
- **ALTER COLUMN子句用于修改原有的列定义，包括修改列名和数据类型**



## 二、修改基本表

**【例】**向Student表增加“入学时间”列，其数据类型为日期型

```
ALTER TABLE Student ADD S_entrance DATE ;
```

**【说明】**不论基本表中原来是否已有数据，新增加的列一律为空值。



## 二、修改基本表

**【例】将年龄的数据类型由字符型（假设原来的数据类型是字符型）改为整数**

```
ALTER TABLE Student ALTER COLUMN Sage INT ;
```

**【例】增加课程名称必须取唯一值的约束条件**

```
ALTER TABLE Course ADD UNIQUE(Cname);
```



### 三、删除基本表

- 语句格式

**DROP TABLE <表名> [ RESTRICT| CASCADE ] ;**

#### 【说明】

- ✓ **RESTRICT**：删除表是有限制的。欲删除的基本表不能被其他表的约束所引用。如果存在依赖该表的对象，则此表不能被删除
- ✓ **CASCADE**：删除该表没有限制。在删除基本表的同时，相关的依赖对象一起删除
- ✓ 基本表定义被删除，数据被删除，表上建立的索引、视图、触发器等一般也将被删除



### 三、删除基本表

**【例】删除Student表**

```
DROP TABLE Student CASCADE ;
```

**【说明】若表上有视图，选择RESTRICT时不能删除，选择CASCADE时表和视图自动被删除**

### 三、删除基本表

**【例】若表上建有视图，选择RESTRICT时表不能删除**

```
CREATE VIEW IS_Student
```

```
AS
```

```
SELECT Sno , Sname , Sage
```

```
FROM Student
```

```
WHERE Sdept='IS' ;
```

```
DROP TABLE Student RESTRICT;
```

```
--ERROR: cannot drop table Student because other objects
depend on it
```

### 三、删除基本表

**【例】** 如果选择CASCADE时可以删除表，视图也自动被删除

```
DROP TABLE Student CASCADE;
```

```
--NOTICE: drop cascades to view IS_Student
```

```
SELECT * FROM IS_Student;
```

```
--ERROR: relation " IS_Student " does not exist
```





# 数据定义



**1 模式的定义与删除**

**2 基本表的定义、删除与修改**

**3 索引的建立与删除**



- **建立索引的目的：加快查询速度**
- **谁可以建立索引**
  - **DBA 或 表的属主（即建立表的人）**
  - **DBMS一般会自动建立以下列上的索引**
    - PRIMARY KEY**
    - UNIQUE**
- **谁维护索引**
  - DBMS自动完成**
- **使用索引**
  - DBMS自动选择是否使用索引以及使用哪些索引**



- RDBMS中索引一般采用B+树、HASH索引来实现
  - B+树索引具有动态平衡的优点
  - HASH索引具有查找速度快的特点
- 采用B+树，还是HASH索引 则由具体的RDBMS来决定
- 索引是关系数据库的内部实现技术，属于内模式的范畴
- CREATE INDEX语句定义索引时，可以定义索引是**唯一索引、非唯一索引或聚簇索引**



## 一、建立索引

- 语句格式

```
CREATE [UNIQUE] [CLUSTER] INDEX <索引名>
ON <表名>(<列名>[<次序>][,<列名>[<次序>]]...);
```

**UNIQUE** 表明此索引的每一个索引值只对应唯一的数据记录。

**CLUSTER** 表示要建立的索引是**聚族索引**。

## 一、建立索引

### (1) 唯一索引 (UNIQUE)

【例】为学生-课程数据库中的Student, Course, SC三个表建立索引。

```
CREATE UNIQUE INDEX Stusno ON Student(Sno);
```

```
CREATE UNIQUE INDEX Coucno ON Course(Cno);
```

```
CREATE UNIQUE INDEX SCno ON SC(Sno ASC, Cno DESC);
```

Student表按学号升序建立唯一索引

Course表按课程号升序建立唯一索引

SC表按学号升序和课程号降序建立唯一索引



## 一、建立索引

(1) 聚簇索引 (CLUSTER)，是指索引顺序与表中记录的物理顺序一致的索引组织

【例】在Student表的Sname (姓名) 列上建立一个聚簇索引

```
CREATE CLUSTER INDEX Stusname
ON Student (Sname);
```

【说明】**修改的是物理存储**

- ✓ 在最经常查询的列上建立聚簇索引以提高查询效率
- ✓ 一个基本表上最多只能建立一个聚簇索引
- ✓ 经常更新的列不宜建立聚簇索引



## 二、删除索引

- 语句格式

**DROP INDEX <索引名> ;**

删除索引时，系统会从**数据字典**中删去有关该索引的描述。

**【例】删除Student表的Stusname索引**

**DROP INDEX Stusname ;**



### 三、数据字典

- ✓ **数据字典是关系数据库管理系统内部的一组系统表**
- ✓ **数据字典记录了数据库所有的定义信息、包括模式定义、视图定义、索引定义、完整性约束定义、各类用户对数据库的权限、统计信息等。**
- ✓ **RDBMS执行SQL数据定义时，实际上就是在更新数据字典**





# SQL语言



3.1 SQL概述

3.2 数据定义

3.3 数据查询

3.4 数据更新

3.5 视图



## • 语句格式

**SELECT** [ALL|DISTINCT] < 目标列表达式 >

[ , <目标列表达式> ] ...

**FROM** <表名或视图名> [ , <表名或视图名> ] ...

[ **WHERE** <条件表达式> ]

[ **GROUP BY** <列名1> [ **HAVING** <条件表达式> ] ]

[ **ORDER BY** <列名2> [ ASC|DESC ] ] ;

### 【注意】

✓语句中的字母不分大小写

✓, ; 等符号是英文状态下的半角符

✓[ ] 中的内容不是必须添加的，为了实现某些功能才添加

- 整个SELECT语句的含义是：根据WHERE子句的条件表达式，从FROM子句指定的基本表或视图找出满足条件的元组，再按SELECT子句中的目标列表表达式，选出元组中的属性值形成结果表。
- 如果有GROUP子句，则将结果按<列名1>的值进行分组，该属性列值相等的元组为一个组，通常会在每组中作用集函数。
- 如果GROUP子句带HAVING短语，则只有满足指定条件的组才予输出。
- 如果有ORDER子句，则结果表还要按<列名2>的值的升序或降序排序。



## 与关系代数的对比

- 在关系表达式：

$\pi_{A_1, A_2, \dots, A_n}(\sigma_F(R_1 \bowtie \dots \bowtie R_m))$

- SQL语句

**SELECT** A1 , A2 , .... An

**FROM** R1  $\bowtie$  ....  $\bowtie$  Rm

**WHERE** F



- 1 单表查询**
- 2 连接查询**
- 3 嵌套查询**
- 4 集合查询**
- 5 基于派生表的查询**
- 6 SELECT 语句的一般格式**



## 单表查询

查询仅涉及一个表，是一种最简单的查询操作

- 选择表中的若干列
- 选择表中的若干元组
- 对查询结果排序
- 使用聚集函数
- 对查询结果分组



## 一、查询指定列

**【例1】查询全体学生的学号与姓名**

```
SELECT Sno , Sname FROM Student ;
```

**【例2】查询全体学生的姓名、学号、所在系**

```
SELECT Sname , Sno , Sdept FROM Student ;
```



## 二、查询全部列

- ✓ 选出所有属性列：
- ✓ 在SELECT关键字后面列出所有列名
- ✓ 将<目标列表表达式>指定为 \*

### 【例3】查询全体学生的详细记录

```
SELECT Sno , Sname , Ssex , Sage , Sdept FROM Student ;
```

或

```
SELECT * FROM Student ;
```





## 三、查询经过计算的值

**SELECT子句的<目标列表达式>可以为：**

- ① 算术表达式
- ② 字符串常量
- ③ 函数
- ④ 列别名



## 三、查询经过计算的值

**【例4】查全体学生的姓名及其出生年份**

```
SELECT Sname , 2005-Sage
/*假定当年的年份为2005年减去学生的年龄*/
FROM Student ;
```

**输出结果：**

| <b>Sname</b> | <b>2005-Sage</b> |
|--------------|------------------|
| <b>李勇</b>    | <b>1984</b>      |
| <b>刘晨</b>    | <b>1985</b>      |
| <b>王敏</b>    | <b>1986</b>      |
| <b>张立</b>    | <b>1985</b>      |



## 三、查询经过计算的值

**【例5】查询全体学生的姓名、出生年份和所有系，要求用小写字母表示所有系名**

```
SELECT Sname , 'Year of Birth: ' , 2005-Sage ,
LOWER(Sdept)
FROM Student ;
```

**输出结果：**

| Sname | 'Year of Birth:' | 2005-Sage | LOWER(Sdept) |
|-------|------------------|-----------|--------------|
| 李勇    | Year of Birth:   | 1984      | cs           |
| 刘晨    | Year of Birth:   | 1985      | is           |
| 王敏    | Year of Birth:   | 1986      | ma           |
| 张立    | Year of Birth:   | 1985      | is           |



## 三、查询经过计算的值

### 【例5】使用列别名改变查询结果的列标题

```
SELECT Sname NAME , 'Year of Birth: ' BIRTH ,
2005-Sage BIRTHDAY , LOWER(Sdept) DEPARTMENT
FROM Student ;
```

输出结果：

| NAME | BIRTH          | BIRTHDAY | DEPARTMENT |
|------|----------------|----------|------------|
| 李勇   | Year of Birth: | 1984     | cs         |
| 刘晨   | Year of Birth: | 1985     | is         |
| 王敏   | Year of Birth: | 1986     | ma         |
| 张立   | Year of Birth: | 1985     | is         |



# 单表查询

查询仅涉及一个表，是一种最简单的查询操作

- 选择表中的若干列
- 选择表中的若干元组
- 对查询结果排序
- 使用聚集函数
- 对查询结果分组



## 一、消除取值重复的行

**【例6】查询选修了课程的学生学号**

```
SELECT Sno FROM SC ;
```

**如果没有指定DISTINCT关键词，  
则缺省为ALL**

**等价于：**

```
SELECT ALL Sno FROM SC ;
```

**执行上面的SELECT语句后，结果为：**

| <u>Sno</u> |
|------------|
| 200215121  |
| 200215121  |
| 200215121  |
| 200215122  |
| 200215122  |



## 一、消除取值重复的行

**【例6】查询选修了课程的学生学号**

**SELECT DISTINCT Sno FROM SC ;** 指定DISTINCT关键词，去掉表中重复的行

**执行上面的SELECT语句后，结果为：**

| Sno       |
|-----------|
| 200215121 |
| 200215122 |



## 二、查询满足条件的元组

### ■使用where子句的常用查询条件

---

| 查 询 条 件    | 谓 词                                          |
|------------|----------------------------------------------|
| 比 较        | =, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符 |
| 确定范围       | <b>BETWEEN AND, NOT BETWEEN AND</b>          |
| 确定集合       | <b>IN, NOT IN</b>                            |
| 字符匹配       | <b>LIKE, NOT LIKE</b>                        |
| 空 值        | <b>IS NULL, IS NOT NULL</b>                  |
| 多重条件（逻辑运算） | <b>AND, OR, NOT</b>                          |

---





## 二、查询满足条件的元组：比较大小

**【例7】查询计算机科学系全体学生的名单**

```
SELECT Sname
FROM Student
WHERE Sdept= 'CS' ;
```

**【例8】查询所有年龄在20岁以下的学生姓名及其年龄**

```
SELECT Sname, Sage
FROM Student
WHERE Sage < 20 ;
```



## 二、查询满足条件的元组：比较大小

**【例9】查询考试成绩有不及格的学生学号**

```
SELECT DISTINCT Sno
FROM SC
WHERE Grade < 60 ;
```



## 二、查询满足条件的元组：确定范围

谓词: **BETWEEN ... AND ...**

**NOT BETWEEN ... AND ...**

**【例10】查询年龄在20~23岁（包括20岁和23岁）之间的学生的姓名、系别和年龄**

```
SELECT Sname , Sdept , Sage
FROM Student
WHERE Sage BETWEEN 20 AND 23 ;
```



## 二、查询满足条件的元组：确定范围

谓词: **BETWEEN ... AND ...**

**NOT BETWEEN ... AND ...**

**【例11】查询年龄不在20~23岁之间的学生姓名、系别和年龄**

**SELECT Sname , Sdept , Sage**

**FROM Student**

**WHERE Sage NOT BETWEEN 20 AND 23 ;**



## 二、查询满足条件的元组：确定集合

谓词：IN <值表>, NOT IN <值表>

【例12】查询信息系（IS）、数学系（MA）和计算机科学系（CS）学生的姓名和性别。

```
SELECT Sname , Ssex
FROM Student
WHERE Sdept IN ('IS' , 'MA' , 'CS');
```



## 二、查询满足条件的元组：确定集合

谓词：IN <值表>， NOT IN <值表>

**【例13】** 查询既不是信息系、数学系，也不是计算机科学系的学生姓名和性别。

```
SELECT Sname , Ssex
FROM Student
WHERE Sdept NOT IN ('IS' , 'MA' , 'CS');
```



## 二、查询满足条件的元组：字符匹配

谓词：[NOT] LIKE ' <匹配串>' [ESCAPE ' <换码字符>' ]

( 1 ) 匹配串为固定字符串

【例14】查询学号为200215121的学生的详细情况。

```
SELECT *
FROM Student
WHERE Sno LIKE '200215121' ;
```

等价于：

```
SELECT *
FROM Student
WHERE Sno = ' 200215121 ' ;
```



## 二、查询满足条件的元组：字符匹配

谓词：[NOT] LIKE ' <匹配串>' [ESCAPE ' <换码字符>' ]

( 2 ) 匹配串为含通配符的字符串

【例15】查询所有姓刘学生的姓名、学号和性别。

```
SELECT Sname , Sno , Ssex
FROM Student
WHERE Sname LIKE '刘%' ;
```

'%' 表达任意长度字符





## 二、查询满足条件的元组：字符匹配

谓词：[NOT] LIKE ' <匹配串>' [ESCAPE ' <换码字符>' ]

( 2 ) 匹配串为含通配符的字符串

【例16】查询姓"欧阳"且全名为三个汉字的学生的姓名。

```
SELECT Sname
FROM Student
WHERE Sname LIKE '欧阳_';
```

'\_' 表达一个字符



## 二、查询满足条件的元组：字符匹配

谓词：[NOT] LIKE ' <匹配串>' [ESCAPE ' <换码字符>' ]

( 2 ) 匹配串为含通配符的字符串

【例17】查询名字中第2个字为"阳"字的学生的姓名和学号。

```
SELECT Sname , Sno
FROM Student
WHERE Sname LIKE '_阳%' ;
```



## 二、查询满足条件的元组：字符匹配

谓词：[NOT] LIKE ' <匹配串>' [ESCAPE ' <换码字符>' ]

( 2 ) 匹配串为含通配符的字符串

【例18】查询所有不姓刘的学生姓名。

```
SELECT Sname , Sno , Ssex
FROM Student
WHERE Sname NOT LIKE '刘%' ;
```



## 二、查询满足条件的元组：字符匹配

谓词：[NOT] LIKE ' <匹配串>' [ESCAPE ' <换码字符>' ]

( 3 ) 使用换码字符将通配符转义为普通字符

【例19】查询DB\_Design课程的课程号和学分。

```
SELECT Cno , Ccredit
```

```
FROM Course
```

```
WHERE Cname LIKE 'DB_Design' ESCAPE '\' ;
```



## 二、查询满足条件的元组：字符匹配

谓词：[NOT] LIKE '**<匹配串>**' [ESCAPE '**<换码字符>**' ]

( 3 ) 使用换码字符将通配符转义为普通字符

**【例20】** 查询以"DB\_"开头，且倒数第3个字符为 i 的课程的具体情况。

```
SELECT *
FROM Course
WHERE Cname LIKE 'DB_%i__' ESCAPE '\' ;
```



## 二、查询满足条件的元组：涉及空值的查询

谓词：IS NULL 或 IS NOT NULL

“IS” 不能用 “=” 代替

【例21】某些学生选修课程后没有参加考试，所以有选课记录，但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号。

```
SELECT Sno , Cno
FROM SC
WHERE Grade IS NULL ;
```



## 二、查询满足条件的元组：涉及空值的查询

谓词：IS NULL 或 IS NOT NULL

“IS” 不能用 “=” 代替

【例22】查所有有成绩的学生学号和课程号。

```
SELECT Sno , Cno
FROM SC
WHERE Grade IS NOT NULL ;
```



## 二、查询满足条件的元组：多重条件查询

逻辑运算符：AND和 OR来联结多个查询条件

- AND的优先级高于OR

- 可以用括号改变优先级

可用来实现多种其他谓词

- [NOT] IN

- [NOT] BETWEEN ... AND ...





## 二、查询满足条件的元组：多重条件查询

**【例23】查询计算机系年龄在20岁以下的学生姓名。**

```
SELECT Sname
FROM Student
WHERE Sdept= 'CS' AND Sage<20 ;
```

## 二、查询满足条件的元组：多重条件查询

【例12（改）】查询信息系（IS）、数学系（MA）和计算机科学系（CS）学生的姓名和性别。

```
SELECT Sname , Ssex
FROM Student
WHERE Sdept IN ('IS' , 'MA' , 'CS ') ;
```

可改写为：

```
SELECT Sname , Ssex
FROM Student
WHERE Sdept= 'IS' OR Sdept= 'MA' OR
Sdept= 'CS ' ;
```



# 单表查询

查询仅涉及一个表，是一种最简单的查询操作

- 选择表中的若干列
- 选择表中的若干元组
- **对查询结果排序**
- 使用聚集函数
- 对查询结果分组



- **ORDER BY子句**
  - **可以按一个或多个属性列排序**
  - **升序：ASC；降序：DESC；缺省值为升序**
- **当排序列含空值时（空值默认为最大值）**
  - **ASC：排序列为空值的元组最后显示**
  - **DESC：排序列为空值的元组最先显示**



**【例24】查询选修了3号课程的学生们的学号及其成绩，查询结果按分数降序排列**

```
SELECT Sno , Grade
FROM SC
WHERE Cno= ' 3 '
ORDER BY Grade DESC ;
```



**【例25】查询全体学生情况，查询结果按所在系的系号升序排列，同一系中的学生按年龄降序排列**

```
SELECT *
FROM Student
ORDER BY Sdept , Sage DESC ;
```



## 单表查询

查询仅涉及一个表，是一种最简单的查询操作

- 选择表中的若干列
- 选择表中的若干元组
- 对查询结果排序
- 使用聚集函数
- 对查询结果分组



- 聚集函数：

- 计数

- COUNT ( [DISTINCT|ALL] \* )

- COUNT ( [DISTINCT|ALL] <列名> )

- 计算总和

- SUM ( [DISTINCT|ALL] <列名> )

- 计算平均值

- AVG ( [DISTINCT|ALL] <列名> )

- 最大最小值

- MAX ( [DISTINCT|ALL] <列名> )

- MIN ( [DISTINCT|ALL] <列名> )





### 【例26】 查询学生总人数

```
SELECT COUNT(*)
FROM Student ;
```

### 【例27】 查询选修了课程的学生人数

```
SELECT COUNT(DISTINCT Sno)
FROM SC ;
```

### 【例28】 计算1号课程的学生平均成绩

```
SELECT AVG(Grade)
FROM SC
WHERE Cno= ' 1 ' ;
```



**【例29】 查询选修1号课程的学生最高分数**

```
SELECT MAX(Grade)
FROM SC
WHERE Cno= '1' ;
```

**【例30】 查询学生200215012选修课程的总学分数**

```
SELECT SUM(Ccredit)
FROM SC , Course
WHER Sno='200215012' AND SC.Cno=Course.Cno;
```



- **聚集函数：**
- **WHERE 子句中是不能使用聚集函数作为条件表达式的。聚集函数只能用于**
- **SELECT子句**
- **和GROUP BY中的HAVING子句。**



## 单表查询

查询仅涉及一个表，是一种最简单的查询操作

- 选择表中的若干列
- 选择表中的若干元组
- 对查询结果排序
- 使用聚集函数
- 对查询结果分组



- **GROUP BY子句分组**：按指定的一列或者多列值分组，值相等的为一组，来细化聚集函数的作用对象。

### 细化聚集函数的作用对象

- 未对查询结果分组，聚集函数将作用于整个查询结果
- 对查询结果分组后，聚集函数将分别作用于每个组



## 【例31】求各个课程号及相应的选课人数

```
SELECT Cno , COUNT(Sno)
FROM SC
GROUP BY Cno ;
```

查询结果：

| Cno | COUNT(Sno) |
|-----|------------|
| 1   | 22         |
| 2   | 34         |
| 3   | 44         |
| 4   | 33         |
| 5   | 48         |

如果有group by, select 后面只能跟group by后面的和聚集函数



### 【例32】 查询选修了3门以上课程的学生学号

```
SELECT Sno
FROM SC
GROUP BY Sno
HAVING COUNT(*) > 3 ;
```

【说明】 HAVING短语与WHERE子句的区别：

- ✓作用对象不同
- ✓WHERE子句作用于基表或视图，从中选择满足条件的元组
- ✓HAVING短语作用于组，从中选择满足条件的组。

**WHERE 子句中不能有聚集函数**



**【例】查询平均成绩大于90分的学生学号和平均成绩  
错误的语句：**

```
SELECT Sno , AVG(Grade) FROM SC
WHERE AVG(Grade) >= 90 GROUP BY Sno ;
```

**正确的查询语句：**

```
SELECT Sno , AVG(Grade) FROM SC
GROUP BY Sno HAVING AVG(Grade) >= 90 ;
```





# SQL语言



3.1 SQL概述

3.2 数据定义

3.3 数据查询

3.4 数据更新

3.5 视图



## 数据查询



- 1 单表查询
- 2 连接查询
- 3 嵌套查询
- 4 集合查询
- 5 基于派生表的查询
- 6 SELECT 语句的一般格式



## 连接查询

同时涉及多个表的查询

连接条件或连接谓词：用来连接两个表的条件

一般格式：

[<表名1>.]<列名1><比较运算符>[<表名2>.]<列名2>

[<表名1>.]<列名1> BETWEEN [<表名2>.]<列名2> AND [<表名2>.]<列名3>

连接字段：连接谓词中的列名称

连接条件中的各连接字段类型必须是可比的，但名字不必是相同的



# 连接查询

1. 当连接运算符为 “=” 称为等值连接，其他运算符称为非等值连接。

2. 连接字段：连接谓词中的列名称

连接条件中的各连接字段类型必须是可比的，但名字不必是相同的



## 连接查询

### 同时涉及多个表的查询

- 等值与非等值连接查询
- 自身连接
- 外连接
- 复合条件连接



## 一、等值与非等值连接查询

等值连接：连接运算符为=

**【例33】** 查询每个学生及其选修课程的情况

```
SELECT Student.* , SC.*
FROM Student , SC
WHERE Student.Sno = SC.Sno ;
```



## 一、等值与非等值连接查询

等值连接：连接运算符为=

【例33】查询每个学生及其选修课程的情况

| Student.Sno | Sname | Ssex | Sage | Sdept | SC.Sno    | Cno | Grade |
|-------------|-------|------|------|-------|-----------|-----|-------|
| 200215121   | 李勇    | 男    | 20   | CS    | 200215121 | 1   | 92    |
| 200215121   | 李勇    | 男    | 20   | CS    | 200215121 | 2   | 85    |
| 200215121   | 李勇    | 男    | 20   | CS    | 200215121 | 3   | 88    |
| 200215122   | 刘晨    | 女    | 19   | CS    | 200215122 | 2   | 90    |
| 200215122   | 刘晨    | 女    | 19   | CS    | 200215122 | 3   | 80    |

## 连接查询

### ■操作的执行过程

- 嵌套循环法
- 排序合并法
- 索引连接

- 首先在表1中找到第一个元组，然后从头开始扫描表2，逐一查找满足连接件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组。
- 表2全部查找完后，再找表1中第二个元组，然后再从头开始扫描表2，逐一查找满足连接条件的元组，找到后就将表1中的第二个元组与该元组拼接起来，形成结果表中一个元组。
- 重复上述操作，直到表1中的全部元组都处理完毕





## 连接查询

### ■ 操作的执行过程

- 嵌套循环法
- 排序合并法
- 索引连接

### 嵌套循环法(NESTED-LOOP)

#### ✓ 基于元组的嵌套循环连接

Result = {}; /\*初始化结果集合\*/

For each tuple s in R

For each tuple r in S

If  $r.B = s.B$  then /\*元组r和元组s满足连接条件\*/

Join r and s as tuple t;

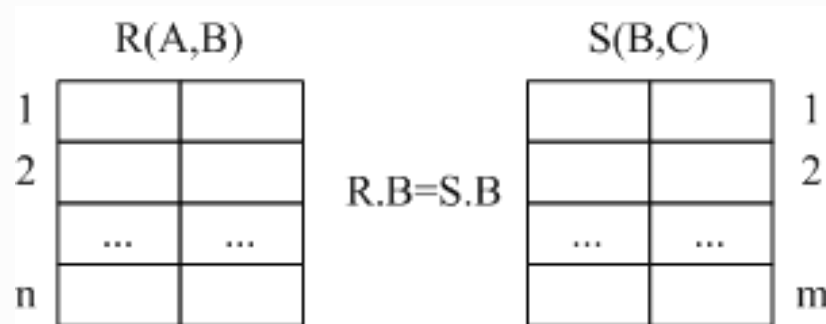
Output t into Result; /\*输出连接结果元组\*/

Endif

Endfor

Endfor

Return Result





## 连接查询

### ■操作的执行过程

- 嵌套循环法
  - 排序合并法
  - 索引连接
- 
- 首先按连接属性对表1和表2排序
  - 对表1的第一个元组，从头开始扫描表2，顺序查找满足连接条件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组。当遇到表2中第一条大于表1连接字段值的元组时，对表2的查询不再继续
  - 找到表1的第二条元组，然后从刚才的中断点处继续顺序扫描表2，查找满足连接条件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组。直接遇到表2中大于表1连接字段值的元组时，对表2的查询不再继续
  - 重复上述操作，直到表1或表2中的全部元组都处理完毕为止



## 连接查询

### ■操作的执行过程

- 嵌套循环法
- 排序合并法
- 索引连接

- 对表2按连接字段建立索引
- 对表1中的每个元组，依次根据其连接字段值查询表2的索引，从中找到满足条件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组
- （该方法可以视作嵌套循环法的一个变种）



## 一、等值与非等值连接查询

等值连接：连接运算符为=

【例34】对[例33]用自然连接完成

```
SELECT Student.Sno , Sname , Ssex , Sage ,
Sdept , Cno , Grade
FROM Student , SC
WHERE Student.Sno = SC.Sno ;
```



## 连接查询

### 同时涉及多个表的查询

- 等值与非等值连接查询
- 自身连接
- 外连接
- 复合条件连接



## 二、自身连接

一个表与其自己进行连接

**【说明】**

- 需要给表起名以示区别
- 由于所有属性名都是同名属性，因此必须使用全名前缀



## 二、自身连接

【例35】查询每一门课的间接先修课（即先修课的先修课）

FIRST表 (Course表)

| Cno | Cname    | Cpno | Ccredit |
|-----|----------|------|---------|
| 1   | 数据库      | 5    | 4       |
| 2   | 数学       |      | 2       |
| 3   | 信息系统     | 1    | 4       |
| 4   | 操作系统     | 6    | 3       |
| 5   | 数据结构     | 7    | 4       |
| 6   | 数据处理     |      | 2       |
| 7   | PASCAL语言 | 6    | 4       |

SECOND表 (Course表)

| Cno | Cname    | Cpno | Ccredit |
|-----|----------|------|---------|
| 1   | 数据库      | 5    | 4       |
| 2   | 数学       |      | 2       |
| 3   | 信息系统     | 1    | 4       |
| 4   | 操作系统     | 6    | 3       |
| 5   | 数据结构     | 7    | 4       |
| 6   | 数据处理     |      | 2       |
| 7   | PASCAL语言 | 6    | 4       |

查询结果：

| Cno | Pcno |
|-----|------|
| 1   | 7    |
| 3   | 5    |
| 5   | 6    |



## 二、自身连接

**【例35】查询每一门课的间接先修课（即先修课的先修课）**

```
SELECT FIRST.Cno , SECOND.Cpno
FROM Course FIRST , Course SECOND
WHERE FIRST.Cpno = SECOND.Cno ;
```





## 连接查询

### 同时涉及多个表的查询

- 等值与非等值连接查询
- 自身连接
- 外连接
- 复合条件连接



### 三、外连接

#### 外连接与普通连接的区别

- 普通连接操作只输出满足连接条件的元组
- 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出

#### 外连接分为：

左外连接：列出左边关系中的所有元组 `LEFT OUT JOIN SC ON`

右外连接：列出右边关系中的所有元组 `LEFT OUT JOIN SC ON`



## 三、外连接

**【例36】改写查询每个学生及其选修课程的情况**

```
SELECT Student.Sno , Sname , Ssex , Sage ,
Sdept , Cno , Grade
FROM Student LEFT OUT JOIN SC ON
(Student.Sno=SC.Sno) ;
```



## 三、外连接

### 【例36】改写查询每个学生及其选修课程的情况

执行结果：

| Student.Sno | Sname | Ssex | Sage | Sdept | Cno  | Grade |
|-------------|-------|------|------|-------|------|-------|
| 200215121   | 李勇    | 男    | 20   | CS    | 1    | 92    |
| 200215121   | 李勇    | 男    | 20   | CS    | 2    | 85    |
| 200215121   | 李勇    | 男    | 20   | CS    | 3    | 88    |
| 200215122   | 刘晨    | 女    | 19   | CS    | 2    | 90    |
| 200215122   | 刘晨    | 女    | 19   | CS    | 3    | 80    |
| 200215123   | 王敏    | 女    | 18   | MA    | NULL | NULL  |
| 200215125   | 张立    | 男    | 19   | IS    | NULL | NULL  |



## 连接查询

### 同时涉及多个表的查询

- 等值与非等值连接查询
- 自身连接
- 外连接
- 复合条件连接



### 四、复合条件连接

#### WHERE子句中含多个连接条件

**【例37】 查询选修2号课程且成绩在90分以上的所有学生**

```
SELECT Student.Sno, Sname
FROM Student, SC
WHERE Student.Sno = SC.Sno AND
/* 连接谓词*/
SC.Cno= '2' AND SC.Grade > 90 ;
/* 其他限定条件 */
```



## 四、复合条件连接

### WHERE子句中含多个连接条件

**【例38】** 查询每个学生的学号、姓名、选修的课程名及成绩

```
SELECT Student.Sno , Sname , Cname , Grade
FROM Student , SC , Course
/*多表连接*/
WHERE Student.Sno = SC.Sno
and SC.Cno = Course.Cno ;
```



## 数据查询



- 1 单表查询
- 2 连接查询
- 3 嵌套查询
- 4 集合查询
- 5 基于派生表的查询
- 6 SELECT 语句的一般格式



## 嵌套查询

- 一个SELECT-FROM-WHERE语句称为一个**查询块**
- 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中的查询称为**嵌套查询**

例如：

```
SELECT Sname /*外层查询/父查询*/
FROM Student
WHERE Sno IN /*内层查询/子查询*/
 (SELECT Sno
 FROM SC
 WHERE Cno='2');
```

```
SELECT Sname
FROM Student,SC
WHERE Student.Sno=SC.Sno
AND Cno='2'
```

- 子查询的限制：不能使用ORDER BY子句,它只能对最终结果排序。
- 层层嵌套方式反映了SQL语言的结构化
- 有些嵌套查询可以用**连接运算**替代



# 嵌套查询

- **不相关子查询：**

**子查询的查询条件不依赖于父查询**

- **由里向外 逐层处理。即每个子查询在上一级查询处理之前求解，子查询的结果用于建立其父查询的查找条件。**



# 嵌套查询

- **相关子查询**：子查询的查询条件依赖于父查询
  - 首先取外层查询中表的第一个元组，根据它与内层查询相关的属性值处理内层查询，若WHERE子句返回值为真，则取此元组放入结果表
  - 然后再取外层表的下一个元组
  - 重复这一过程，直至外层表全部检查完为止



# 嵌套查询

- 带有IN谓词的子查询
- 带有比较运算符的子查询
- 带有ANY ( SOME ) 或ALL谓词的子查询
- 带有EXISTS谓词的子查询



## 一、带有IN谓词的子查询

**【例39】** 查询与“刘晨”在同一个系学习的学生。

此查询要求可以分步来完成

① 确定“刘晨”所在系名

```
SELECT Sdept
FROM Student
```

```
WHERE Sname= '刘晨' ;
```

结果为：CS



## 一、带有IN谓词的子查询

**【例39】** 查询与“刘晨”在同一个系学习的学生。

此查询要求可以分步来完成

② 查找所有在CS系学习的学生。

```
SELECT Sno , Sname , Sdept
FROM Student
WHERE Sdept= ' CS ' ;
```

结果为：

| Sno       | Sname | Sdept |
|-----------|-------|-------|
| 200215121 | 李勇    | CS    |
| 200215122 | 刘晨    | CS    |



### 一、带有IN谓词的子查询

**【例39】** 查询与“刘晨”在同一个系学习的学生。

此查询要求可以分步来完成

将第一步查询嵌入到第二步查询的条件中

```
SELECT Sno , Sname , Sdept
```

```
FROM Student
```

```
WHERE Sdept IN
```

```
(SELECT Sdept
```

```
FROM Student
```

此查询为**不相关子查询**。 WHERE Sname= ‘刘晨’ );



用自身连接完成[例39]查询要求

SELECT S1.Sno,S1.Sname,S1.Sdept

FROM Student S1 , Student S2

WHERE S1.Sdept = S2.Sdept AND

S2.Sname = '刘晨' ;



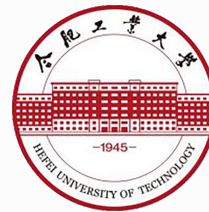


## 一、带有IN谓词的子查询

**【例40】** 查询选修了课程名为“信息系统”的学生学号和姓名

```
SELECT Sno, Sname
FROM Student
WHERE Sno IN
 (SELECT Sno
 FROM SC
 WHERE Cno IN
 (SELECT Cno
 FROM Course
 WHERE Cname='信息系统'
 ;))
```

- ③ 最后在Student关系中取出Sno和Sname
- ② 然后在SC关系中找到选修了3号课程的学生学号
- ① 首先在Course关系中找到“信息系统”的课程号,为3号



**用连接查询实现[例40]**

**SELECT Sno , Sname**

**FROM Student , SC , Course**

**WHERE Student.Sno = SC.Sno AND**

**SC.Cno = Course.Cno AND**

**Course.Cname='信息系统' ;**



# 嵌套查询

- 带有IN谓词的子查询
- 带有比较运算符的子查询
- 带有ANY ( SOME ) 或ALL谓词的子查询
- 带有EXISTS谓词的子查询



## 二、带有比较运算符的子查询

- 当能确切知道内层查询返回单值时，可用比较运算符（>，<，=，>=，<=，!=或<>）。
- 与ANY或ALL谓词配合使用

例：假设一个学生只可能在一个系学习，并且必须属于一个系，则在[例39]可以用 = 代替IN：

```
SELECT Sno , Sname , Sdept
FROM Student
WHERE Sdept =
 (SELECT Sdept
 FROM Student
 WHERE Sname= '刘晨');
```



## 二、带有比较运算符的子查询

**\* 子查询一定要跟在比较符之后**

**错误**的例子：

```
SELECT Sno , Sname , Sdept
FROM Student
WHERE (SELECT Sdept
 FROM Student
 WHERE Sname= ' 刘晨 ')
 = Sdept ;
```



## 二、带有比较运算符的子查询

**【例41】找出每个学生超过他选修课程平均成绩的课程号**

```
SELECT Sno , Cno
FROM SC x
WHERE Grade >=(SELECT AVG(Grade)
FROM SC y
WHERE y.Sno=x.Sno);
```



## 二、带有比较运算符的子查询

执行过程：

1. 从外层查询中取出SC的一个元组x，将元组x的Sno值（200215121）传送给内层查询。

```
SELECT AVG(Grade)
FROM SC y
WHERE y.Sno='200215121';
```

2. 执行内层查询，得到值88，用该值代替内层查询，得到外层查询：

```
SELECT Sno , Cno
FROM SC x
WHERE Grade >=88 ;
```



## 二、带有比较运算符的子查询

执行过程：

3. 执行这个查询，得到

( 200215121 , 1 )

( 200215121 , 3 )

4. 外层查询取出下一个元组重复做上述1至3步骤，取第二个学号，第三个学号，直到外层的SC元组全部处理完毕。结果为：

( 200215121 , 1 )

( 200215121 , 3 )

( 200215122 , 2 )





# 嵌套查询

- 带有IN谓词的子查询
- 带有比较运算符的子查询
- 带有**ANY或ALL**谓词的子查询
- 带有EXISTS谓词的子查询



### 三、带有ANY ( SOME ) 或ALL谓词的子查询

#### 谓词语义

■ANY : 任意一个值

■ALL : 所有值



## 三、带有ANY或ALL谓词的子查询

需要配合使用比较运算符

- > ANY 大于子查询结果中的某个值
- > ALL 大于子查询结果中的所有值
- < ANY 小于子查询结果中的某个值
- < ALL 小于子查询结果中的所有值
- >= ANY 大于等于子查询结果中的某个值
- >= ALL 大于等于子查询结果中的所有值
- <= ANY 小于等于子查询结果中的某个值
- <= ALL 小于等于子查询结果中的所有值
- = ANY 等于子查询结果中的某个值
- = ALL 等于子查询结果中的所有值（通常没有实际意义）
- != (或<>) ANY 不等于子查询结果中的某个值
- != (或<>) ALL 不等于子查询结果中的任何一个值



## 三、带有ANY或ALL谓词的子查询

**【例42】** 查询其他系中比计算机科学某一学生年龄小的学生姓名和年龄

```
SELECT Sname , Sage
FROM Student
WHERE Sage < ANY (SELECT Sage
 FROM Student
 WHERE Sdept= ' CS ')
AND Sdept <> 'CS' ; /*父查询块中的条件 */
```



## 三、带有ANY或ALL谓词的子查询

结果：

| Sname | Sage |
|-------|------|
| 王敏    | 18   |
| 张立    | 19   |

执行过程：

1. RDBMS执行此查询时，首先处理子查询，找出CS系中所有学生的年龄，构成一个集合(20, 19)
2. 处理父查询，找所有不是CS系且年龄小于20 或 19的学生



## 三、带有ANY或ALL谓词的子查询 用**聚集函数**实现【例42】

```
SELECT Sname , Sage
FROM Student
WHERE Sage <
 (SELECT MAX(Sage)
 FROM Student
 WHERE Sdept= 'CS ')
AND Sdept <> ' CS ' ;
```



## 三、带有ANY或ALL谓词的子查询

【例43】查询其他系中比计算机科学系**所有**学生年龄都小的学生姓名及年龄

方法一：用ALL谓词

```
SELECT Sname , Sage
FROM Student
WHERE Sage < ALL
 (SELECT Sage
 FROM Student
 WHERE Sdept= ' CS ')
AND Sdept <> ' CS ' ;
```



## 三、带有ANY或ALL谓词的子查询

【例43】查询其他系中比计算机科学系**所有**学生年龄都小的学生姓名及年龄

方法二：用聚集函数

```
SELECT Sname , Sage
```

```
FROM Student
```

```
WHERE Sage <
```

```
(SELECT MIN(Sage)
```

```
FROM Student
```

```
WHERE Sdept= ' CS ')
```

```
AND Sdept <>' CS ' ;
```





## 三、带有ANY或ALL谓词的子查询

**ANY ( 或SOME ) , ALL谓词与聚集函数、IN谓词的等价转换关系**

|     | =  | <>或!=  | <    | <=     | >    | >=     |
|-----|----|--------|------|--------|------|--------|
| ANY | IN | --     | <MAX | <=MAX  | >MIN | >= MIN |
| ALL | -- | NOT IN | <MIN | <= MIN | >MAX | >= MAX |



# 嵌套查询

- 带有IN谓词的子查询
- 带有比较运算符的子查询
- 带有ANY ( SOME ) 或ALL谓词的子查询
- 带有EXISTS谓词的子查询



## 四、带有EXISTS谓词的子查询

### • 1.EXISTS谓词

- 存在量词 $\exists$
- 带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”。
  - 若内层查询结果非空，则外层的WHERE子句返回真值
  - 若内层查询结果为空，则外层的WHERE子句返回假值
- 由EXISTS引出的子查询，其目标列表表达式通常都用\*，因为带EXISTS的子查询只返回真值或假值，给出列名无实际意义

### • 2.NOT EXISTS谓词

- 若内层查询结果非空，则外层的WHERE子句返回假值
- 若内层查询结果为空，则外层的WHERE子句返回真值



### 四、带有EXISTS谓词的子查询

**【例44】查询所有选修了1号课程的学生姓名**

**思路分析：**

- 本查询涉及Student和SC关系
- 在Student中依次取每个元组的Sno值，用此值去检查SC关系
- 若SC中存在这样的元组，其Sno值等于此Student.Sno值，并且其Cno= '1'，则取此Student.Sname送入结果关系



## 四、带有EXISTS谓词的子查询

**【例44】查询所有选修了1号课程的学生姓名  
用嵌套查询**

```
SELECT Sname
FROM Student
WHERE EXISTS
 (SELECT *
 FROM SC
 WHERE Sno=Student.Sno AND Cno= '1') ;
```



# 数据查询



Student

| 学号<br>Sno | 姓名<br>Sname | 性别<br>Ssex | 年龄<br>Sage | 系科<br>Sdept |
|-----------|-------------|------------|------------|-------------|
| 95001     | 李勇          | 男          | 20         | CS          |
| 95002     | 刘晨          | 女          | 19         | IS          |
| 95003     | 王敏          | 女          | 18         | MA          |
| 95004     | 张立          | 男          | 19         | IS          |

SC

| Sno   | Cno | Grade |
|-------|-----|-------|
| 95001 | 1   | 92    |
| 95001 | 2   | 85    |
| 95001 | 3   | 88    |
| 95002 | 2   | 90    |
| 95002 | 3   | 80    |

SELECT Sname  
FROM Student  
WHERE EXISTS

(SELECT \*  
FROM SC  
WHERE Sno=Student.Sno AND Cno='1');

True

SELECT \*  
FROM SC  
WHERE Sno=Student.Sno AND Cno='1'

95001

李勇

95001

1

92



## 四、带有EXISTS谓词的子查询

**【例44】查询所有选修了1号课程的学生姓名  
用连接运算**

```
SELECT Sname
FROM Student, SC
WHERE Student.Sno=SC.Sno AND
SC.Cno= '1';
```



## 四、带有EXISTS谓词的子查询

**【例45】查询没有选修1号课程的学生姓名**

```
SELECT Sname
FROM Student
WHERE NOT EXISTS
 (SELECT *
 FROM SC
 WHERE Sno = Student.Sno AND Cno='1') ;
```





### 四、带有EXISTS谓词的子查询

**【例】【例39】** 查询与“刘晨”在同一个系学习的学生  
可以用带EXISTS谓词的子查询替换：

```
SELECT Sno , Sname , Sdept
FROM Student S1
WHERE EXISTS
 (SELECT *
 FROM Student S2
 WHERE S2.Sdept = S1.Sdept AND
 S2.Sname = '刘晨');
```



## 四、带有EXISTS谓词的子查询

- 不同形式的查询间的替换

- 一些带EXISTS或NOT EXISTS谓词的子查询不能被其他形式的子查询等价替换
- 所有带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换

- 用EXISTS/NOT EXISTS实现全称量词(难点)

SQL语言中没有全称量词 $\forall$  ( For all )

可以把带有全称量词的谓词转换为等价的带有存在量词的谓词：

$$(\forall x)P \equiv \neg (\exists x(\neg P))$$



## 四、带有EXISTS谓词的子查询

等价于：没有一门课程是他不选修的

### 【例46】查询选修了全部课程的学生姓名

```
SELECT Sname
FROM Student
WHERE NOT EXISTS
```

不存在这样的  
课程记录

```
(SELECT *
 FROM Course
 WHERE NOT EXISTS
```

```
(SELECT *
 FROM SC
```

该学生没有选  
修的课程记录

```
 WHERE Sno= Student.Sno
```

```
 AND Cno= Course.Cno)
```

```
) ;
```



# 数据查询



Student

| 学号<br>Sno | 姓名<br>Sname | 性别<br>Ssex | 年龄<br>Sage | 系科<br>Sdept |
|-----------|-------------|------------|------------|-------------|
| 95001     | 李勇          | 男          | 20         | CS          |
| 95002     | 刘晨          | 女          | 19         | IS          |
| 95003     | 王敏          | 女          | 18         | MA          |
| 95004     | 张立          | 男          | 19         | IS          |

SC

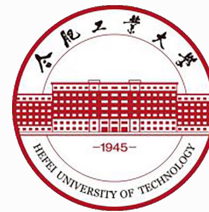
| 学号<br>Sno | 课程号<br>Cno | 成绩<br>Grade |
|-----------|------------|-------------|
| 95001     | 1          | 92          |
| 95001     | 2          | 85          |
| 95001     | 3          | 88          |
| 95002     | 2          | 90          |
| 95002     | 3          | 80          |

Course

| 课程号<br>Cno | 课程名<br>Cname | 先行课<br>Cpno | 学分<br>Ccredit |
|------------|--------------|-------------|---------------|
| 1          | 数据库          | 5           | 4             |
| 2          | 数学           |             | 2             |
| 3          | 信息系统         | 1           | 4             |
| 4          | 操作系统         | 6           | 3             |
| 5          | 数据结构         | 7           | 4             |
| 6          | 数据处理         |             | 2             |
| 7          | C语言          | 6           | 4             |

**SELECT Sname FROM Student** → 95001  
**WHERE NOT EXISTS**  
**(SELECT \* FROM Course** → (4, 5, 6, 7)  
**WHERE NOT EXISTS**  
**(SELECT \* FROM SC**  
**WHERE Sno=Student.Sno AND**  
**Cno=Course.Cno);**

查找出没有选修的课



## 四、带有EXISTS谓词的子查询

### 用EXISTS/NOT EXISTS实现逻辑蕴涵(难点)

- SQL语言中没有蕴涵(Implication)逻辑运算
- 可以利用谓词演算将逻辑蕴涵谓词等价转换为：

$$p \rightarrow q \equiv \neg p \vee q$$



### 四、带有EXISTS谓词的子查询

**【例47】查询至少选修了学生200215122选修的全部课程的学生号码**

**解题思路：**

**用逻辑蕴涵表达：查询学号为x的学生，对所有的课程y，只要200215122学生选修了课程y，则x也选修了y。**

**形式化表示：**

**用P表示谓词 “学生200215122选修了课程y”**

**用q表示谓词 “学生x选修了课程y”**

**则上述查询为:  $(\forall y) p \rightarrow q$**



## 四、带有EXISTS谓词的子查询

- 等价变换：

$$\begin{aligned}(\forall y)p \rightarrow q &\equiv \neg (\exists y (\neg(p \rightarrow q))) \\ &\equiv \neg (\exists y (\neg(\neg p \vee q))) \\ &\equiv \neg \exists y(p \wedge \neg q)\end{aligned}$$

- 变换后语义：不存在这样的课程y，学生200215122选修了y，而学生x没有选。



## 四、带有EXISTS谓词的子查询

- 用NOT EXISTS谓词表示：

```
SELECT DISTINCT Sno
FROM SC SCX
WHERE NOT EXISTS
```

不存在这样的  
课程记录

```
(SELECT *
FROM SC SCY
WHERE SCY.Sno = ' 200215122 ' AND
NOT EXISTS
(SELECT *
FROM SC SCZ
WHERE SCZ.Sno=SCX.Sno AND
SCZ.Cno=SCY.Cno));
```

200215122选了，但是  
该学生没有选





## 数据查询



**SCX**

| 学号<br>Sno | 课程号<br>Cno | 成绩<br>Grade |
|-----------|------------|-------------|
| 15121     | 1          | 92          |
| 15121     | 2          | 85          |
| 15121     | 3          | 88          |
| 15122     | 2          | 90          |
| 15122     | 3          | 80          |

**SCY**

| 学号<br>Sno | 课程号<br>Cno | 成绩<br>Grade |
|-----------|------------|-------------|
| 15121     | 1          | 92          |
| 15121     | 2          | 85          |
| 15121     | 3          | 88          |
| 15122     | 2          | 90          |
| 15122     | 3          | 80          |

**SCZ**

| 学号<br>Sno | 课程号<br>Cno | 成绩<br>Grade |
|-----------|------------|-------------|
| 15121     | 1          | 92          |
| 15121     | 2          | 85          |
| 15121     | 3          | 88          |
| 15122     | 2          | 90          |
| 15122     | 3          | 80          |

**SELECT DISTINCT Sno FROM SC**

**WHERE NOT EXISTS**

**(SELECT \* FROM SC**

**WHERE SCY.Sno='200215122' AND**

**NOT EXISTS**

**(SELECT \* FROM SC**

**WHERE SCZ.Sno=SCX.Sno AND SCZ.Cno=SCY.Cno);**

对于每一个学生都要检查：他不存在  
这样的情况，200215122选修了的课  
程，而这个学生不存在选修



# SQL语言



3.1 SQL概述

3.2 数据定义

3.3 数据查询

3.4 数据更新

3.5 视图



## 数据查询



- 1 单表查询
- 2 连接查询
- 3 嵌套查询
- 4 集合查询
- 5 基于派生表的查询
- 6 SELECT 语句的一般格式



# 集合查询

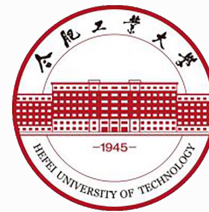
- 集合操作的种类
  - 并操作 **UNION**
  - 交操作 **INTERSECT**
  - 差操作 **EXCEPT**
- 参加集合操作的各查询结果的列数必须相同；对应项的数据类型也必须相同

# 集合查询

**【例48】查询计算机科学系的学生及年龄不大于19岁的学生**  
**方法一：**

```
SELECT *
FROM Student
WHERE Sdept= 'CS'
UNION
SELECT *
FROM Student
WHERE Sage<=19 ;
```

- **UNION**：将多个查询结果合并起来时，系统自动去掉重复元组。
- **UNION ALL**：将多个查询结果合并起来时，保留重复元组



## 集合查询

**【例48】查询计算机科学系的学生及年龄不大于19岁的学生**

**方法二：**

```
SELECT DISTINCT *
FROM Student
WHERE Sdept= 'CS' OR Sage<=19 ;
```



## 集合查询

**【例49】 查询选修了课程1或者选修了课程2的学生**

```
SELECT Sno
FROM SC
WHERE Cno=' 1 '

UNION

SELECT Sno
FROM SC
WHERE Cno= ' 2 ' ;
```



## 集合查询

**【例50】 查询计算机科学系的学生与年龄不大于19岁的学生的交集**

```
SELECT *
FROM Student
WHERE Sdept='CS'
INTERSECT
SELECT *
FROM Student
WHERE Sage<=19
```





# 集合查询

**【例50】查询计算机科学系的学生与年龄不大于19岁的学生的交集  
实际上就是查询计算机科学系中年龄不大于19岁的学生**

```
SELECT *
FROM Student
WHERE Sdept= 'CS' AND Sage<=19 ;
```



# 集合查询

**【例51】 查询选修课程1的学生集合与选修课程2的学生集合的交集**

```
SELECT Sno
FROM SC
WHERE Cno='1 '
INTERSECT
SELECT Sno
FROM SC
WHERE Cno='2 ' ;
```



## 集合查询

**【例51】 查询选修课程1的学生集合与选修课程2的学生集合的交集  
实际上是查询既选修了课程1又选修了课程2的学生**

```
SELECT Sno
FROM SC
WHERE Cno=' 1 ' AND Sno IN
 (SELECT Sno
 FROM SC
 WHERE Cno=' 2 ') ;
```



## 集合查询

**【例52】查询计算机科学系的学生与年龄不大于19岁的学生的差集**

```
SELECT *
FROM Student
WHERE Sdept='CS'
EXCEPT
SELECT *
FROM Student
WHERE Sage <=19;
```



# 集合查询

**【例52】查询计算机科学系的学生与年龄不大于19岁的学生的差集  
实际上是查询计算机科学系中年龄大于19岁的学生**

```
SELECT *
FROM Student
WHERE Sdept= 'CS' AND Sage>19 ;
```



## 数据查询



- 1 单表查询
- 2 连接查询
- 3 嵌套查询
- 4 集合查询
- 5 基于派生表的查询
- 6 SELECT 语句的一般格式



# 基于派生表的查询

子查询不但能够出现在**Where**子句中，也能够出现在**from**子句中，作为一个**临时表**成为主查询的对象。



## 基于派生表的查询

**【例53】** 找出每个学生超过他自己选修课程平均成绩的课程号

**方法一：** Select Sno, Cno  
From SC x  
Where Grade >= (select avg(Grade)  
From SC y where y.Sno=x.Sno);

**方法二：** Select Sno, Cno  
from SC , (select Sno,avg(Grade)  
from SC group by Sno )  
**AS Avg\_sc**(avg\_sno,avg\_grade)  
where SC.Sno=Avg\_sc.avg\_sno and  
SC.grade>=Avg\_sc.avg\_grade;





## 数据查询



- 1 单表查询
- 2 连接查询
- 3 嵌套查询
- 4 集合查询
- 5 基于派生表的查询
- 6 **SELECT 语句的一般格式**



## SELECT 语句的一般格式

**SELECT** [ALL|DISTINCT]

<目标列表达式> [别名] [ , <目标列表达式> [别名]] ...

**FROM** <表名或视图名> [别名]

[ , <表名或视图名> [别名]] ...

[**WHERE** <条件表达式>]

[**GROUP BY** <列名1>[ , <列名1'>] ...

[**HAVING** <条件表达式>]

[**ORDER BY** <列名2> [ASC|DESC] [ , <列名2'> [ASC|DESC] ] ... ] ;



有如下三个关系表，请完成相应操作：

**练习：有如下三个关系表，请完成相应操作：**

SC：学生选课成绩表

| SNO | CNO | GRADE |
|-----|-----|-------|
| 1   | K1  | 83    |
| 2   | K1  | 85    |
| 3   | K1  | 92    |
| 2   | K5  | 90    |
| 3   | K5  | 84    |
| 3   | K8  | 80    |

S：学生信息表

| SNO | SNAME | AGE | SEX |
|-----|-------|-----|-----|
| 1   | 李强    | 23  | 男   |
| 2   | 刘丽    | 22  | 女   |
| 3   | 张友    | 22  | 男   |

C：课程信息表

| CNO<br>(课号) | CNAME<br>(课名) | TEACHER<br>(教师) |
|-------------|---------------|-----------------|
| k1          | C语言           | 王华              |
| k5          | 数据库原理         | 程军              |
| k8          | 编译原理          | 程军              |



- (1)检索“程军”老师所授课程的课程号CNO和课程名CNAME。
- (2)检索年龄大于21的男学生学号SNO和姓名SNAME。
- (3)检索选修课程名为“C语言”的学生学号和姓名。
- (4)检索“李强”同学不学课程的课程号。
- (5)检索选修课程包含“程军”老师所授课程之一的学生学号。
- (6)检索选修课程号为k1和k5的学生学号。
- (7)检索至少选修“程军”老师所授全部课程的学生姓名SNAME。
- (8)检索全部学生都选修的课程的课程号和课程名。



(1) 检索“程军”老师所授课程的课程号CNO和课程名CNAME。

```
SELECT CNO,CNAME
FROM C
WHERE TEACHER='程军' ;
```

(2) 检索年龄大于21的男学生学号SNO和姓名SNAME。

```
SELECT SNO,SNAME
FROM S
WHERE AGE>21 AND SEX='男' ;
```



(3)检索选修课程名为“C语言”的学生学号和姓名。

```
SELECT S.SNO,SNAME
FROM S,SC,C
WHERE S.SNO=SC.SNO AND C.CNO=SC.CNO AND
CNAME='C语言' ;
```

(4)检索“李强”同学不学课程的课程号。

```
SELECT CNO
FROM C
WHERE CNO NOT IN
(SELECT CNO
FROM SC,S
WHERE SC.SNO=S.SNO AND S.SNAME='李强');
```



(5)检索选修课程包含“程军”老师所授课程之一的学生学号。

```
SELECT SNO
FROM C, SC
WHERE C.CNO=SC.CNO AND C.TEACHER='程军' ;
```

```
SELECT SNO
FROM S
WHERE EXISTS
 (SELECT *
 FROM SC, C
 WHERE C.CNO=SC.CNO AND C.TEACHER='程军')
```

(6)检索选修课程号为k1和k5的学生学号。

```
SELECT A.SNO
FROM SC A, SC B
WHERE A.SNO=B.SNO AND
 A.CNO='k1' AND B.CNO='k5' ;
```

```
SELECT SNO
FROM SC
WHERE CNO='k1' AND
 SNO IN (SELECT SNO
 FROM SC
 WHERE CNO='k5');
```



(7)检索至少选修“程军”老师所授全部课程的学生姓名SNAME。

**SELECT SNAME**      **等价于：不存在一门程军老师的课是该学生没选的**  
**FROM S**  
**WHERE NOT EXISTS**  
**(SELECT \***  
**FROM C**  
**WHERE TEACHER='程军' AND NOT EXISTS**  
**(SELECT \***  
**FROM SC**  
**WHERE SC.SNO=S.SNO AND SC.CNO=C.CNO));**





(8)检索全部学生都选修的课程的课程号和课程名。

**SELECT CNO,CNAME**

**FROM C**

**等价于：不存在一个学生是没有选这门课的**

**WHERE NOT EXISTS**

**(SELECT \***

**FROM S**

**WHERE NOT EXISTS**

**(SELECT \***

**FROM SC**

**WHERE SC.CNO=C.CNO AND SC.SNO=S.SNO));**



# SQL语言



3.1 SQL概述

3.2 数据定义

3.3 数据查询

3.4 数据更新

3.5 视图



## 数据更新

- 1 插入数据
- 2 修改数据
- 3 删除数据





# 插入数据

- **两种插入数据方式**
  1. **插入元组**
  2. **插入子查询结果**
    - **可以一次插入多个元组**



# 一、插入元组

- 语句格式

INSERT

INTO <表名> [(<属性列1>[ , <属性列2 >...])]

VALUES (<常量1> [ , <常量2>] ... )

- 功能

- 将新元组插入指定表中



# 一、插入元组

## • INTO子句

- 属性列的顺序可与表定义中的顺序不一致
- 没有指定属性列
- 指定部分属性列

## • VALUES子句

- 提供的值必须与INTO子句匹配
  - 值的个数
  - 值的类型



# 一、插入元组

**【例1】** 将一个新学生元组（学号：200215128；姓名：陈冬；性别：男；所在系：IS；年龄：18岁）插入到Student表中

**INSERT**

**INTO Student (Sno , Sname , Ssex , Sdept , Sage)**

**VALUES ('200215128' , '陈冬' , '男' , 'IS' , 18) ;**



# 一、插入元组

**【例2】将学生张成民的信息插入到Student表中**

```
INSERT
 INTO Student
 VALUES ('200215126' , '张成民' , '男' , 18 , 'CS');
```





# 一、插入元组

**【例3】插入一条选课记录( '200215128' , '1 ' )**

```
INSERT
 INTO SC(Sno , Cno)
 VALUES (' 200215128 ' , ' 1 ');
```

**RDBMS将在新插入记录的Grade列上自动地赋空值。**

或者：

```
INSERT
 INTO SC
 VALUES (' 200215128 ' , ' 1 ' , NULL) ;
```



## 二、插入子查询结果

- 语句格式

**INSERT**

**INTO <表名> [( <属性列1> [ , <属性列2>... )]**

**子查询 ;**

- 功能

**将子查询结果插入指定表中**



## 二、插入子查询结果

- INTO子句(与插入元组类似)
- 子查询
  - SELECT子句目标列必须与INTO子句匹配
    - 值的个数
    - 值的类型



## 二、插入子查询结果

**【例4】对每一个系，求学生的平均年龄，并把结果存入数据库**

第一步：建表

```
CREATE TABLE Dept_age
(Sdept CHAR(15), /* 系名*/
 Avg_age SMALLINT) ; /*学生平均年龄*/
```



## 二、插入子查询结果

**【例4】对每一个系，求学生的平均年龄，并把结果存入数据库**

第二步：插入数据

```
INSERT
INTO Dept_age(Sdept , Avg_age)
SELECT Sdept , AVG(Sage)
FROM Student
GROUP BY Sdept ;
```

## 二、插入子查询结果

RDBMS在执行插入语句时会检查所插元组是否破坏表上已定义的完整性规则

- 实体完整性
- 参照完整性
- 用户定义的完整性
  - NOT NULL约束
  - UNIQUE约束
  - 值域约束



## 数据更新

- 1 插入数据
- 2 修改数据
- 3 删除数据





# 修改数据

- 语句格式

**UPDATE <表名>**

**SET <列名>=<表达式>[ , <列名>=<表达式>]...**

**[WHERE <条件>];**

- 功能

- 修改指定表中满足WHERE子句条件的元组





# 修改数据

## ■ SET子句

- 指定修改方式
- 要修改的列
- 修改后取值

## ■ WHERE子句

- 指定要修改的元组
- 缺省表示要修改表中的所有元组



# 修改数据

- **三种修改方式**

1. **修改某一个元组的值**
2. **修改多个元组的值**
3. **带子查询的修改语句**



## 修改数据

### 一、修改某一个元组的值

**【例5】** 将学生200215121的年龄改为22岁

```
UPDATE Student
```

```
SET Sage=22
```

```
WHERE Sno=' 200215121 ' ;
```



## 修改数据

### 二、修改多个元组的值

**【例6】** 将所有学生的年龄增加1岁

```
UPDATE Student

SET Sage= Sage+1 ;
```

# 修改数据

## 三、带子查询的修改语句

**【例7】** 将计算机科学系全体学生的成绩置零

```
UPDATE SC
SET Grade=0
WHERE 'CS'=
 (SELETE Sdept
 FROM Student
 WHERE Student.Sno= SC.Sno) ;
```



# 修改数据

**RDBMS在执行修改语句时会检查修改操作  
是否破坏表上已定义的完整性规则**

- **实体完整性**
- **主码不允许修改**
- **用户定义的完整性**
  - **NOT NULL约束**
  - **UNIQUE约束**
  - **值域约束**



## 数据更新

- 1 插入数据
- 2 修改数据
- 3 删除数据



# 删除数据

- 语句格式

**DELETE**

**FROM <表名>**

**[WHERE <条件>] ;**

- 功能

- 删除指定表中满足WHERE子句条件的元组

- WHERE子句

- 指定要删除的元组
- 缺省表示要删除表中的全部元组，表的定义仍在字典中（删除数据和删除表的区别之一）





# 删除数据

- **三种删除方式**
  1. **删除某一个元组的值**
  2. **删除多个元组的值**
  3. **带子查询的删除语句**



# 删除数据

## 一、删除某一个元组的值

**【例8】删除学号为200215128的学生记录**

**DELETE**

**FROM Student**

**WHERE Sno= 200215128 ' ;**



# 删除数据

## 二、删除多个元组的值

**【例9】删除所有的学生选课记录**

```
DELETE

FROM SC ;
```



# 删除数据

## 三、带子查询的删除语句

**【例10】删除所有的学生选课记录**

```
DELETE
FROM SC
WHERE 'CS'=
 (SELETE Sdept
FROM Student
WHERE Student.Sno=SC.Sno) ;
```



## 空值的处理

**空值的存在是因为取值有不确定性，对关系运算带来特殊问题。**

**该属性有值，但当前不知道**

**该属性不应该有值**

**由于某种原因不便于填写**

## 空值的处理

**NULL的不确定性 “不知道” “不存在” “无意义”**

**【例11】向SC表中插入一个元组，学生号是“201215126”，课程号是“1”，成绩为空**

```
INSERT
INTO SC(Sno, Cno, Grade)
VALUES('201215126', '1', NULL);
```

**或**

```
VALUES('201215126', '1');
```



## 空值的处理

**NULL的不确定性 “不知道” “不存在” “无意义”**

**【例12】将Student表中学生号为 “201215200”的学生所属的系改为空值**

```
UPDATE Student
SET Sdept=NULL
WHERE Sno= '201215200' ;
```

## 空值的处理

**空值的判断:判断一个属性的值是否为空值，用IS NULL或IS NOT NULL来表示**

**【例】SELECT\* FROM Student**

**WHERE Sname IS NULL OR Ssex IS NULL OR Sage IS NULL OR  
Sdept IS NULL ;**

**空值的约束条件**

- **属性定义（或者域定义）中有NOT NULL约束条件的不能取空值，加了UNIQUE限制的属性不能取空值，码属性不能取空值**





## 空值的处理

**算术运算:空值与另一个值的算术运算结果为空值**

**比较运算:空值与另一个值的算术运算结果为UNKNOWN**

**逻辑运算:**

表3.8 逻辑运算符真值表

| x | y | x AND y | x OR y | NOT x |
|---|---|---------|--------|-------|
| T | T | T       | T      | F     |
| T | U | U       | T      | F     |
| T | F | F       | T      | F     |
| U | T | U       | T      | U     |
| U | U | U       | U      | U     |
| U | F | F       | U      | U     |
| F | T | F       | T      | T     |
| F | U | F       | U      | T     |
| F | F | F       | F      | T     |

T表示TRUE, F表示FALSE, U表示UNKNOWN



## 空值的处理

**【例13】找出选修1号课程不及格的学生学号**

```
SELECT Sno
```

```
FROM SC
```

```
WHERE Grade<60 AND Cno='1'
```

以上查询**不包括**缺考学生，缺考学生的Grade为NULL，返回UNKNOWN



## 空值的处理

**【例13】找出选修1号课程不及格的学生学号**

```
SELECT Sno
FROM SC
WHERE Grade<60 AND Cno='1'
UNION
SELECT Sno
FROM SC
WHERE Grade IS NULL AND Cno='1'
```



# 空值的处理

**【例13】找出选修1号课程不及格的学生学号**

```
SELECT Sno
FROM SC
WHERE AND Cno='1' AND (Grade<60 OR Grade
IS NULL)
```



# SQL语言



3.1 SQL概述

3.2 数据定义

3.3 数据查询

3.4 数据更新

3.5 视图



## 视图的特点

- 虚表，是从一个或几个基本表（或视图）导出的表
- 只存放视图的定义，不存放视图对应的数据
- 基本表中的数据发生变化，从视图中查询出的数据也随之改变



# 视图

- 1 定义视图
- 2 查询视图
- 3 更新视图
- 4 视图的作用



## 创建视图

- 语句格式

**CREATE VIEW**

**<视图名> [(<列名> [, <列名>]...)]**

**AS <子查询>**

**[WITH CHECK OPTION] ;**

- 组成视图的属性列名：全部省略或全部指定
- 子查询不允许含有ORDER BY子句和DISTINCT短语



# 创建视图

- **RDBMS执行CREATE VIEW语句时只是把视图定义存入数据字典，并不执行其中的SELECT语句。**
- **在对视图查询时，按视图的定义从基本表中将数据查出。**

# 创建视图

**【例1】 建立信息系学生的视图。**

```
CREATE VIEW IS_Student
AS
SELECT Sno , Sname , Sage
FROM Student
WHERE Sdept= 'IS' ;
```

## 创建视图

**【例2】建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生。**

```
CREATE VIEW IS_Student
```

```
AS
```

```
SELECT Sno , Sname , Sage
```

```
FROM Student
```

```
WHERE Sdept= 'IS'
```

```
WITH CHECK OPTION ; (即子查询中的条件表达式)
```

■ **WITH CHECK OPTION**

**透过视图进行增删改操作时，不得破坏视图定义中的谓词条件**



## 创建视图

上例中加入了**WITH CHECK OPTION**，所有DBMS对IS\_Student视图的更新操作：

- 修改操作：自动加上Sdept= 'IS'的条件
- 删除操作：自动加上Sdept= 'IS'的条件
- 插入操作：自动检查Sdept属性值是否为'IS'
  - 如果不是，则拒绝该插入操作
  - 如果没有提供Sdept属性值，则自动定义Sdept为'IS'

## 创建视图

**【例3】建立信息系选修了1号课程的学生视图。**

```
CREATE VIEW IS_S1(Sno , Sname , Grade)
```

```
AS
```

```
SELECT Student.Sno , Sname , Grade
```

```
FROM Student , SC
```

```
WHERE Sdept= 'IS' AND
```

```
Student.Sno=SC.Sno AND
```

```
SC.Cno= '1' ;
```

基于**多个基表**的视图

## 创建视图

**【例4】建立信息系选修了1号课程且成绩在90分以上的学生的视图。**

```
CREATE VIEW IS_S2
```

```
AS
```

```
SELECT Sno , Sname , Grade
```

```
FROM IS_S1
```

```
WHERE Grade>=90 ;
```

基于视图的视图

## 创建视图

【例5】定义一个反映学生出生年份的视图。

```
CREATE VIEW BT_S(Sno , Sname , Sbirth)
```

```
AS
```

带表达式的视图

```
SELECT Sno , Sname , 2005-Sage
```

```
FROM Student ;
```

# 创建视图

**【例6】** 将学生的学号及他的平均成绩定义为一个视图。

```
CREAT VIEW S_G(Sno , Gavg)
```

```
AS
```

分组视图

```
SELECT Sno , AVG(Grade)
```

```
FROM SC
```

```
GROUP BY Sno ;
```



## 创建视图

【例7】将Student表中所有女生记录定义为一个视图。

```
CREATE VIEW F_Student (F_Sno , name , sex , age , dept)
AS
SELECT * 不指定属性列
FROM Student
WHERE Ssex='女' ;
```

➤ 不易扩充的视图,以 SELECT \* 方式创建的视图可扩充性差，应尽可能避免



## 删除视图

- 语句的格式：

**DROP VIEW <视图名> ;**

- 该语句从数据字典中删除指定的视图定义
- 如果该视图上还导出了其他视图，使用**CASCADE**级联删除语句，把该视图和由它导出的所有视图一起删除
- 删除基表时，由该基表导出的所有视图定义都必须显式地使用**DROP VIEW**语句删除

# 删除视图

**【例8】删除视图IS\_S1**

**DROP VIEW IS\_S1 ;**

**IS\_S2 视图导出了IS\_S2所以该语句拒绝执行**

**DROP VIEW IS\_S1 **CASCADE** ;**



# 视图

- 1 定义视图
- 2 查询视图
- 3 更新视图
- 4 视图的作用



# 查询视图

- 用户角度：查询视图与查询基本表相同
- RDBMS实现视图查询的方法
  - 视图消解法 ( View Resolution )
    - 进行有效性检查
    - 转换成等价的对基本表的查询
    - 执行修正后的查询

## 查询视图

**【例9】在信息系学生的视图中找出年龄小于20岁的学生**

```
SELECT Sno , Sage
FROM IS_Student
WHERE Sage<20 ;
```

**视图消解转换后的查询语句为：**

```
SELECT Sno , Sage
FROM Student
WHERE Sdept= 'IS' AND Sage<20 ;
```

## 查询视图

**【例10】 查询选修了1号课程的信息系学生**

```
SELECT IS_Student.Sno , Sname
FROM IS_Student , SC
WHERE IS_Student.Sno =SC.Sno AND SC.Cno= '1' ;
```

## 查询视图

在某些情况下，视图消解法不能生成正确查询。

**【例11】在S\_G视图中查询平均成绩在90分以上的学生学号和平均成绩**

```
SELECT *
FROM S_G
WHERE Gavg>=90 ;
```

**S\_G子查询如下：**

```
CREATE VIEW S_G (Sno , Gavg)
AS
SELECT Sno , AVG(Grade)
FROM SC
GROUP BY Sno ;
```



## 查询视图

【例11】在S\_G视图中查询平均成绩在90分以上的学生学号和平均成绩

错误：

```
SELECT Sno , AVG(Grade)
FROM SC
WHERE AVG(Grade)>=90
GROUP BY Sno ;
```

正确：

```
SELECT Sno , AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade)>=90 ;
```

视图消解法的局限  
有些情况下，视图  
消解法不能生成正  
确查询。



# 视图

- 1 定义视图
- 2 查询视图
- 3 更新视图
- 4 视图的作用



# 更新视图

**【说明】**视图更新是通过视图进行插入、删除、修改数据，因为视图是不存储数据的，因此对视图的更新操作是通过视图消解，转换为对实际表的数据更新操作。

**【注意】**为了防止更新视图出错，实际使用时要加上WITH CHECK OPTION语句

## 更新视图

**【例12】将信息系学生视图IS\_Student中学号200215122的学生姓名改为“刘辰”**

```
UPDATE IS_Student
SET Sname= '刘辰'
WHERE Sno= ' 200215122 ' ;
```

**转换后的语句：**

```
UPDATE Student
SET Sname= '刘辰'
WHERE Sno= ' 200215122 ' AND Sdept= 'IS' ;
```

## 更新视图

**【例13】向信息系学生视图IS\_S中插入一个新的学生记录：200215129，赵新，20岁**

```
INSERT
INTO IS_Student
VALUES('95029', '赵新', 20);
```

**转换为对基本表的更新：**

```
INSERT
INTO Student(Sno, Sname, Sage, Sdept)
VALUES('200215129', '赵新', 20, 'IS');
```

# 更新视图

**【例14】删除信息系学生视图IS\_Student中学号为200215129的记录**

**DELETE**

**FROM IS\_Student**

**WHERE Sno= ' 200215129 ' ;**

**转换为对基本表的更新：**

**DELETE**

**FROM Student**

**WHERE Sno= ' 200215129 ' AND Sdept= 'IS' ;**

## 更新视图

**更新视图的限制：一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新**

**例：视图S\_G为不可更新视图。**

```
UPDATE S_G
```

```
SET Gavg=90
```

```
WHERE Sno= '200215121' ;
```

**这个对视图的更新无法转换成对基本表SC的更新，系统无法修改各科成绩，是平均分到90**



# 视图

- 1 定义视图
- 2 查询视图
- 3 更新视图
- 4 视图的作用





## 视图的作用

1. 视图能够简化用户的操作；
2. 视图使用户能以多种角度看待同一数据；
3. 视图对重构数据库提供了一定程度的逻辑独立性；
  - 由于对视图的更新是有条件的，因此应用程序中修改数据的语句可能仍会因基本表结构的改变而改变。
4. 视图能够对机密数据提供安全保护；
  - ✓ 对不同用户定义不同视图，使每个用户只能看到他有权看到的数据
  - ✓ 通过WITH CHECK OPTION对关键数据定义操作限制
5. 适当的利用视图可以更清晰的表达查询。



# ■SQL：结构化查询语言

- DDL
  - Create SCHEMA
  - Create Table
- DML
  - Insert...Into
  - Update...Set...Where
  - Delete...from...where
  - Select...From...where

单表查询 多表查询  
模糊查询 结果排序  
结果去重复 条件书写



# 作业

长江雨课堂：作业2：SQL语句练习